



*Write code from scratch in a
clear & concise way, with a
complete basic course.*

*From beginners to
intermediate.*

Python for Beginners with applications

Lesson 5:
Creating and using functions
File Handling
Error & Exception handling
Numpy library

Abdesselam Filali 2025-04-23 18h00 Algiers time

1.

Creating and using functions



```
1
2 A function is a block of code that performs a specific task when it is
3 called, and functions are reusable, which means you can write a function
4 once and use it as many times as you want.
5
6 Python has several built-in functions that you may be familiar with,
7 including:
8 - print() prints an object to the terminal
9 - int()   converts a string or number data type to an integer data type
10 - len()   returns the length of an object
11
12 Function names include parentheses and may include parameter (arguments).
13 we'll go over how to define your own functions to use in your coding
14 projects
```

1 A function is defined by using the `def` keyword, followed by a name of
2 your choosing, followed by a set of parentheses which hold any
3 parameters the function will take (they can be empty), and ending with
4 a colon.

```
1  def hello():  
2      print("Hello, World!")
```

8 Our function is now fully defined, but if we run the program at this
9 point, nothing will happen since we didn't call the function.

```
10  
11  
12  
13  def hello():  
14      print("Hello, World!")  
15      hello()
```

Level 1 : basic function without any parameter

1_01_funcLevel1.py > ...

```
1  def my_function():                # creating a simple function
2      print("Hello from a function")
3
4  my_function()                     # calling the function
5
```

Level 2 : function with parameters

1_01_funcLevel2Param.py > ...

```
1 def greet1(name):    # This function greets to the person passed in as a parameter
2     print(f"Hello, {name}")
3 greet1("Ahmed")
4
5 def greet2(name, age):    # This function has 2 parameters
6     print(f"Hello, {name}! You are {age} years old.")
7 greet2("Amine", 25)
```

1 Level 3 : return one or more values

2
3  1_01_funcLevel3ReturnValues.py > ...

```
4     1  def calculate(a,b):  
5       2      |  return a+b, a-b, a*b, a/b  
6       3  print(calculate(10, 20))
```

Level 4 : default parameter values, parameter order

1_01_funcLevel4DefaultParam.py > ...

```
1 def greet3(name, age=25):    # This function has 2 parameters, one by default
2     print(f"Hello, {name}! You are {age} years old.")
3 greet3("Asmaa")
4 greet3("Asmaa", 27)
5
6 def greet4(name, age):    # This function has 2 parameters
7     print(f"Hello, {name}! You are {age} years old.")
8 greet4(age=24, name="Reema")    # changing arguments order
```


1 Level 5 : docstring

2  1_01_funcLevel5Docstring.py > ...

3 1 def divide(x, y):

4 2 """

5 3 Divide x by y and return the result.

6 4 :param x: numerator

7 5 :param y: denominator

8 6 :return: division result

9 7 """

10 8 return x / y

11 9 result = divide(10, 2)

12 10 print(result)

13

14

Level 6 : variable scope

 1_01_funcLevel6VarScope.py > ...

```
1  global_var = 10
2  def some_function():
3      local_var = 5
4      print(global_var + local_var)
5  some_function()
```

Level 7 : recursion

1_01_funcLevel7Recursion.py > ...

```
1 def factorial(n):  
2     if n == 0:  
3         return 1  
4     else: return n * factorial(n - 1)  
5  
6 print(factorial(10))
```

1 Level 8 : Lambda

2  1_01_funcLevel8Lambda.py > ...

3 1 VAT=0.19

4 2 VAT_Included = lambda x: x * (1 + VAT)

5 3 print(VAT_Included(100))

6

7

8

9

10

11

12

13

14

1 Level 9 : Arbitrary positional parameters

```
2 1_01_funcLevel9ArbitraryPosition.py > ...  
3  
4 1 def sum_all(*args):  
5     """ Sum all given arguments. """  
6     return sum(args)  
7 total = sum_all(1, 2, 3, 4, 5)  
8 print(total)  
9  
10  
11  
12  
13  
14
```

1 Level 10 : functional

2
3  1_01_funcLevel9Functional.py > ...

4 1 def apply(func, x):
5 2 return func(x)

6
7 4 def square(x):
8 5 return x ** 2

9
10 7 result = apply(square, 5)
11 8 print(result)

12
13
14

Level 11 : using the functools library

Run the following codes (adding 2 lines of code, line 2 and 3)

The time library is used to calculate the execution time

1_01_funcLevel11usingFTools.py > ...

```
1 import time
2 def fib(n):
3     if n <= 1:
4         return n
5     else:
6         return fib(n - 1) + fib(n - 2)
7 start=time.time()
8 result=fib(40)
9 print(f"Time: {time.time() - start:.10f} s")
10 print(result)
11
12
```

1_01_funcLevel11usingFTools.py > ...

```
1 import time
2 from functools import lru_cache
3 @lru_cache(maxsize=None)
4 def fib(n):
5     if n <= 1:
6         return n
7     else:
8         return fib(n - 1) + fib(n - 2)
9 start=time.time()
10 result=fib(40)
11 print(f"Time: {time.time() - start:.10f} s")
12 print(result)
```

1 Python Program to Count the Number of Vowels in a String

2  1_02_example.py > ...

3 1 # Python Program to Count the Number of Vowels in a String

4 2 def count_vowels(input_string):

5 3 | vowels = "aeiouAEIOU"

6 4 | count = 0

7 5 | for char in input_string:

8 6 | | if char in vowels:

9 7 | | | count += 1

10 8 | return count

11 9 user_input = input("Enter a string: ")

12 10 vowel_count = count_vowels(user_input)

13 11 print(f"The number of vowels in the string is: {vowel_count}")

14

1 Python Program to Find the GCD of Two Numbers

2  1_02_example2.py >  find_gcd

3 1 def find_gcd(a, b):

4 2 while b:

5 3 a, b = b, a % b

6 4 return a

7 5 num1 = int(input("Enter the first number: "))

8 6 num2 = int(input("Enter the second number: "))

9 7 gcd = find_gcd(num1, num2)

10 8 print(f"The GCD of {num1} and {num2} is {gcd}.")

11

12

13

14

1 Logical of a number

2
3  1_02_example3.py > ...

4 1 def logical(x):

5 2 if x == 1:

6 3 return 1

7 4 else:

8 5 return(x + logical(x-1))

9 6 result = logical(5)

10 7 print(result)

11

12

13

14

1 Scope

```
2 1_02_example4.py > ...  
3 1 def fun():  
4   |     x = 100  
5   |     print(x)  
6 4 x+=1  
7 5 fun()  
8  
9  
10  
11  
12  
13  
14
```

Scope

1_02_example5.py > ...

```
1 x=10
2 def modify_x():
3     x=x+1
4     print(x)
5 modify_x()
```

1_02_example5.py > ...

```
1 x = 10
2 def modify_x(num):
3     return num + 1
4 x = modify_x(x)
5 print(x)
```

1 Scope

1_02_example5.py > ...

```
1 x=10
2 def modify_x():
3     x=x+1
4     print(x)
5 modify_x()
```

1_02_example5.py > ...

```
1 x = 10
2 def modify_x(num):
3     return num + 1
4 x = modify_x(x)
5 print(x)
```

2.

File handling in Python



File handling is an important part of any application.

Python has several functions for creating, reading, updating, and deleting files.

تعد معالجة الملفات جزءاً مهماً من أي تطبيق.

لدى Python العديد من الدوال لإنشاء الملفات وقراءتها وتحديثها وحذفها.



```
1 The key function for working with files in Python is the open() function.
2 The open() function takes two parameters; filename, and mode.
3 There are four different methods (modes) for opening a file:
4
5
6 "r" - Read - Default value. Opens a file for reading, error if the file does not
7       exist
8 "a" - Append - Opens a file for appending, creates the file if it does not exist
9 "w" - Write - Opens a file for writing, creates the file if it does not exist
10 "x" - Create - Creates the specified file, returns an error if the file exists
11
12 In addition, you can specify if the file should be handled as binary or text mode
13 "t" - Text - Default value. Text mode
14 "b" - Binary - Binary mode (e.g. images)
```


Filename	Mode	Reading Method
"test.txt"	"rt"	read()

Code:

```
2_01_openFile_read.py > ...  
1  myFile = open("test1.txt", 'rb')  
2  print(myFile.read())  
3  myFile.close()  
4  myFile.read()
```

Because "r" for read and "t" for text are the default values, you don't need to specify them.

```
1
2   Read a file:
3
4   - The file is in the same directory
5   - The file is in a different directory
6   - The file doesn't exist
7   - Read and print it on screen
8
9
10
11
12
13
14
```

Complete path

Code:

```
2_04_openFile_read.py > ...  
1  f = open("D://myfiles/welcome.txt", "r")  
2  print(f.read())  
3
```

Slash instead of back slash

Read part of a file

Filename

"test.txt"

Mode

"rt"

Reading Method

Read(n)

Code:

 2_02_openFile_read.py > ...

```
1  # Return the 6 first characters of the file
2  f = open("test.txt", "r")
3  print(f.read(6))
```

1
2
3 Read an image file

- 4
- 5 - Al-Aqsa.jpg (in the same directory)
 - 6 - Binary file
 - 7 - Read and print it on screen
 - 8 - Show the image
 - 9 - Use a library to manipulate the image file
- 10
11
12
13
14

Read part of a file (lines)

Filename

Mode

Reading Method

"test.txt"

"rt"

Readline(n)

Code:

```
2_03_openFile_readline.py > ...  
1  myFile = open("test.txt", 'rt')  
2  print(myFile.readline())  
3  print(myFile.readline())  
4  print(myFile.readline())  
5  for x in myFile:  
6      | print(x, end='')  
7  myFile.close()
```

1 To write to an existing file, you must add a parameter to
2 the `open()` function:

3 "a" - Append - will append to the end of the file

4 "w" - Write - will overwrite any existing content

Code: 2_05_writeToFile.py > ...

```
5 1 f = open("test.txt", "a")
6 2 f.write("\nNow the file has more content!")
7 3 f.close()
8 4 #open and read the file after the appending:
9 5 f = open("test.txt", "r")
10 6 print(f.read())
11 7
12 8 f = open("file1.txt", "w")
13 9 f.write("Woops! I have deleted the content!")
14 10 f.close()
15 11
16 12 #open and read the file after the overwriting:
17 13 f = open("demofile3.txt", "r")
18 14 print(f.read())
```

1 To create a file, you must add a parameter to the `open()` function:
2 "x" - Create a new file
3 "w" - Create a new file id it doesn't exist

7 Code: 2_06_createFile.py > ...

```
1 f = open("file1.txt", "x") # Create a new file
2 f = open("file2.txt", "w") # Create a new file if it doesn't exist
```


1 To delete a file, you must use the `os` library

2

3 `os.remove(file.txt)` - Remove a file

3

4 `os.rmdir(folder1)` - Remove a directory, you can only delete an empty one

4

5 Code:

2_07_deleteFile.py > ...

6

1 `import os`

7

2 `os.remove("file2.txt")`

8

3

9

4 `if os.path.exists("file3.txt"):`

10

5 | `os.remove("file3.txt")`

11

6 `else:`

12

7 | `print("The file does not exist")`

13

14

Writing to a file

Filename

"test.txt"

Mode

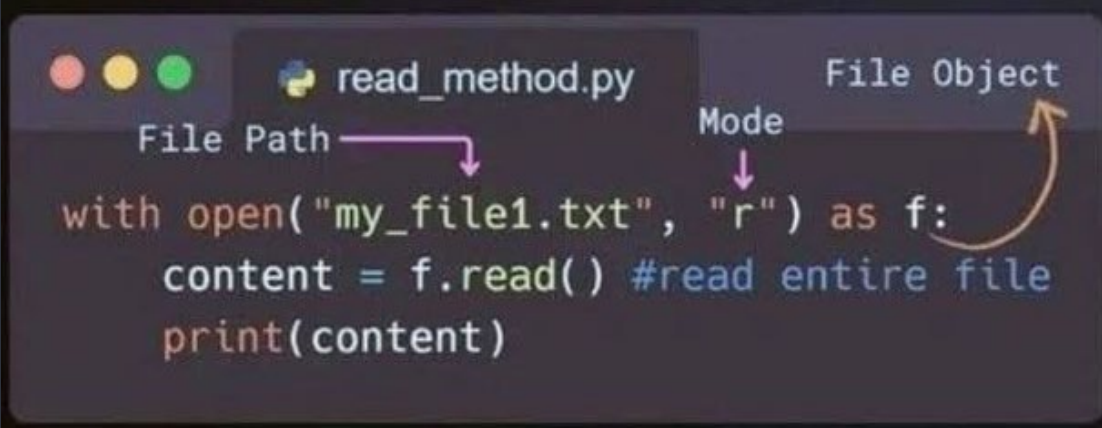
"rt"

Reading Method

read()

Code:

```
2_1_openFile_read.py > ...  
1  myFile = open("test.txt", 'rt')  
2  print(myFile.read())
```

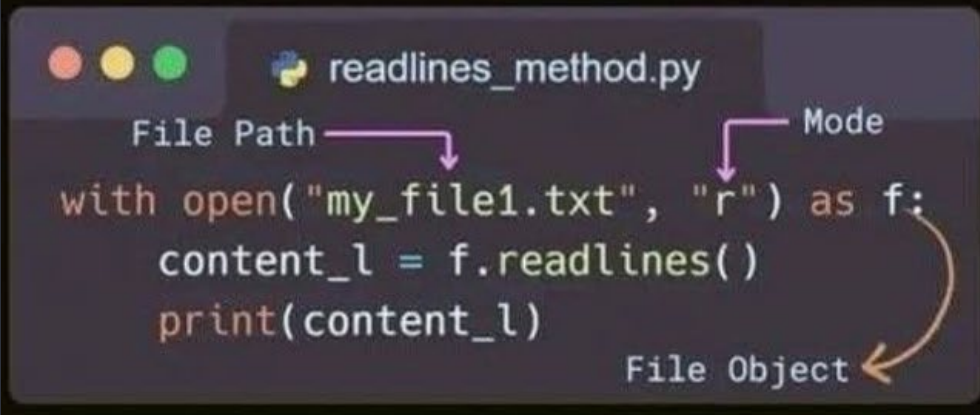


```
1  read_method.py
2
3  File Path
4
5  with open("my_file1.txt", "r") as f:
6      content = f.read() #read entire file
7      print(content)
```

The `read()` method reads a specified number of characters (or bytes) from a file, or the entire file content if no size is specified.

The `readline()` method in Python is employed to sequentially read one line at a time from a file.

```
readline_method.py  
File Path  
with open("my_file1.txt", "r") as f:  
    line1 = f.readline()  
    print(line1)  
Mode  
File Object
```



```
1  readlines_method.py
2
3  File Path
4  with open("my_file1.txt", "r") as f:
5      content_l = f.readlines()
6      print(content_l)
7
8  Mode
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

The `readlines()` method reads all lines from a file, returning them as a list where each element corresponds to a line in the file.

3.

Exceptions and Error handling



ZeroDivisionError is raised when a program attempts to perform a division operation where the denominator is zero.

Syntax: `ZeroDivisionError: division by zero`

Why :

Direct Division by Zero

Code:  3_01_exceptionHandling.py > ...

```
1  numerator = 10
2  denominator = 0
3  result = numerator / denominator
4  print(result)
```

Why : Variable Initialization Issue**Code:** 3_01_exceptionHandling.py > [🔗] denominator

```
1 denominator = 0
```

```
2 result = 20 / denominator
```

Why : Conditional Statements Issue**Code:** 3_01_exceptionHandling.py > ...

```
1 denominator = 0
```

```
2 if denominator != 0:
```

```
3     result = 25 / denominator
```


Approach/Reason to Solve Zerodivisionerror

Use if-else

Code:  3_01_exceptionHandlingSol.py > ...

```
1  numerator = 10
2  denominator = 0
3
4  if denominator != 0:
5      result = numerator / denominator
6  else:
7      print("Error: Cannot divide by zero")
```

Using Conditional Expression

1 3_01_exceptionHandlingSol.py > ...

2 1 numerator = 10

3 2 denominator = 0

4 3

4 4 result = numerator / denominator if denominator != 0 else "Error: Denominator cannot be zero"

5 5 print(result)

Catching the Exception (Using Try-Except Block)

8 3_01_exceptionHandlingSol.py > ...

9 1 numerator = 10

10 2 denominator = 0

11 3 try:

12 4 | result = numerator / denominator

13 5 except ZeroDivisionError:

14 6 | print("Error: Cannot divide by zero")

Input a valid number

3_01_exceptionHandlingSolExample.py > ...

```
1 while True:
2     try:
3         x = int(input("Please enter a number: "))
4         break
5     except ValueError:
6         print("Oops! That was no valid number. Try again...")
7 print(x)
```

Input a valid number

3_01_exceptionHandlingSolExample.py > ...

```
1 while True:
2     try:
3         x = int(input("Please enter a number: "))
4         break
5     except ValueError:
6         print("Oops! That was no valid number. Try again...")
7 print(x)
```

Code:

```
3_02_errorHandling.py > ...  
1 # bad practice  
2 def read_file(file_path):  
3     file = open(file_path, 'r')  
4     content = file.read()  
5     file.close()  
6     return content  
7 print(read_file('textError.txt'))  
8  
9 # good practice  
10 def read_file(file_path):  
11     try:  
12         with open(file_path, 'r') as file:  
13             return file.read()  
14     except FileNotFoundError:  
15         return "File not found."  
16 print(read_file('text_Error.txt'))
```

4.

NumPy Library



BREAKING NEWS



What is NumPy?

NumPy (Numerical Python) is a fundamental powerful library for mathematical and numerical operations. It provides support for large, multi-dimensional arrays and matrices, along with a wide range of mathematical functions to operate on these arrays efficiently.

Why use NumPy?

- Much faster than Python lists for numerical tasks
- Used as the foundation for libraries like Pandas, TensorFlow, and Scikit-learn
- Makes matrix operations, statistics, and data transformations easy and efficient

installation

```
pip install numpy
```



What is NumPy Array?

A NumPy array is a powerful data structure that stores elements of the same data type in a grid-like format. It's much faster and more efficient than Python lists for numerical operations..

Creating Arrays:

```
import numpy as np

np.array([1, 2, 3])           # From a list
np.zeros((2, 3))              # Array of zeros
np.ones((3, 3))               # Array of ones
np.arange(0, 10, 2)           # Range of numbers
```

Creating Array:**Code:**

```
4_01_numpy.py > ...
1  import numpy as np
2  arr = np.array([1, 2, 3, 4, 5]) # Creating a NumPy array
3  result = arr * 2 # Performing mathematical
4  |         |         |         |         | # operations on the array
5  print(result)      #output [2 4 6 8 10]
```

Basic Array Operations:**Indexing and slicing:**

```
arr = np.array([10, 20, 30, 40])  
arr[1]          # 20  
arr[1:3]        # [20 30]
```

Array Shape & Dimensions:

```
arr.shape        # Returns (rows, columns)  
arr.ndim         # Returns number of dimensions  
arr.size         # Total number of elements
```

Reshaping Arrays:

```
arr = np.array([[1, 2], [3, 4], [5, 6]])  
arr.reshape(2, 3) # Change the shape
```

Reshaping Arrays:

```
arr.dtype        # Get the data type of elements
```

Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])
```

```
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

Reshaping Arrays:

Mathematical Operations:**Element-Wise operations:**

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
a + b      # [5 7 9]  
a * b      # [4 10 18]  
a ** 2     # [1 4 9]
```

Aggregate functions:

```
arr = np.array([5, 10, 15])  
  
np.sum(arr)      # 30  
np.mean(arr)     # 10.0  
np.max(arr)      # 15  
np.std(arr)      # Standard Deviation
```

NumPy makes math easy – no need for loops! Just write it once and it works on the entire array.

Let's use NumPy in a real-world scenario: analyzing temperature data!
Example: Calculate Average Weekly Temperature

```
4_05_CalculateAvgWeeklyTemp.py > ...  
1  import numpy as np  
2  
3  # temperature in degrees celsius for 7 days  
4  temps=np.array([29.3, 42.1, 18.8, 22.1, 17.6, 28.7, 33.4])  
5  
6  #average temperature  
7  avg_temp=np.mean(temps)  
8  print(f"Average temperature for 7 days: {avg_temp:.1f} degrees celsius")  
9  
10 #highest temperature  
11 max_temp=np.max(temps)  
12 print(f"Highest temperature for 7 days: {max_temp:.1f} degrees celsius")  
13  
14 #lowest temperature  
15 min_temp=np.min(temps)  
16 print(f"Lowest temperature for 7 days: {min_temp:.1f} degrees celsius")
```

#1 **np.array()**: Create a NumPy array from a Python list or tuple.

#2 **np.zeros()**: Create an array filled with zeros of a specified shape.

#3 **np.ones()**: Create an array filled with ones of a specified shape.

#4 **np.arange()**: Create an array with values within a specified range.

#5 **np.linspace()**: Create an array with evenly spaced values over a specified interval.

#6 **np.reshape()**: Reshape an array into a new shape.

#7 **np.random.rand()**: The rand() function is used to generate an array with random values between 0 to 1.

#8 **np.random.randn()**: Generate an array of random numbers from a standard normal distribution. (close to zero)

#9 **np.random.randint()**: Generate an array of random integers within a specified range.

#10 **np.mean()**: Calculate the mean of array elements.

#11 **np.median()**: Calculate the median of array elements.

- #12 **np.std()**: Calculate the standard deviation of array elements.
- #13 **np.sum()**: Compute the sum of array elements.
- #14 **np.min(), np.max()**: Find the minimum and maximum values in an array.
- #15 **np.argmin(), np.argmax()**: Find the indices of the minimum and maximum values in an array.
- #16 **np.dot()**: Compute the dot product of two arrays.
- #17 **np.transpose()**: Calculate transpose of the array.
- #18 **np.concatenate()**: Concatenate arrays along a specified axis.

#19 **np.split()**: Split an array into multiple sub-arrays.

#20 **np.vstack()**, **np.hstack()**: Stack arrays vertically and horizontally.

#21 **np.unique()**: Find the unique elements in an array.

#22 **np.save()**, **np.load()**: Save and load arrays to/from disk.

#23 **np.clip()**: Clip (limit) the values in an array.

#24 **np.where()**: Return elements chosen from two arrays based on a condition.

#25 **np.linalg.inv()**: Calculate inverse of the matrix.

- #26 **np.linalg.det()**: Calculate determinant of the matrix.
- #27 **np.linalg.solve()**: Solve a system of linear equations.
- #28 **np.percentile()**: Compute the nth percentile of the data.
- #29 **np.corrcoef()**: Compute the correlation coefficient between two arrays.
- #30 **np.deg2rad()**, **np.rad2deg()**: Convert angles from degrees to radians and vice versa.
- #31 **np.argsort()**: Return the indices that would sort an array.
- #32 **np.searchsorted()**: Find indices where elements should be inserted to maintain order.

#33 `np.arcsin()`, `np.arccos()`, `np.arctan()`:
Inverse trigonometric functions.

#34 `np.linalg.eig()`: Eigenvalues and
eigenvectors of a square matrix.

#35 `np.linalg.svd()`: Singular value
decomposition.